

Manual de Documentación Técnica y de Usuario — Dance Labo (LABITO)

Sistema Inteligente de Captura, Análisis y Evaluación de Movimientos de Baile

1. Introducción y Propósito del Sistema

El proyecto **Dance Labo** (conocido en código como **LABITO**) nace con la finalidad de automatizar y democratizar la enseñanza artística y los procesos de rehabilitación física mediante el uso de inteligencia artificial. Tradicionalmente, la captura de movimiento óptico requería laboratorios costosos y trajes con marcadores reflexivos. Dance Labo propone una solución interactiva de captura monocular libre de marcadores (*markerless capture*) utilizando una cámara web convencional.

El sistema realiza tres funciones principales de forma integrada:

- Captura y Retransmisión (Capture Studio):** Extrae los movimientos de una persona frente a la cámara RGB en tiempo real y los replica tridimensionalmente sobre un avatar esquelético en Unity.
- Modelado Secuencial (Machine Learning):** Entrena modelos de redes neuronales profundas que aprenden la dinámica espaciotemporal de una danza.
- Evaluación de Desempeño (Live Evaluation):** Compara el baile del usuario frente a la referencia ideal frame a frame, calculando desviaciones geométricas y enviando feedback interactivo por colores en Unity.

2. Justificación Técnica del Ecosistema (Tech Stack)

Para garantizar un rendimiento estable y preciso a tiempo real (30 FPS) en computadoras comerciales comunes sin GPU dedicada, se diseñó un pipeline de visión híbrido y ligero:

Herramienta	Función Principal	Razón de Elección y Ventajas
YOLOv8 (Ultralytics)	Detección de Personas y RoI	Evita falsos positivos y ruido de fondo. Al recortar la silueta del usuario antes del análisis esquelético, incrementa masivamente la inmunidad al ruido visual ambiental.
MediaPipe Pose (Google)	Estimación de Landmarks 3D	Genera los 33 landmarks corporales con coordenadas (x, y, z) de forma ágil sobre CPU. Su integración posterior al recorte de YOLO optimiza su estabilidad.
MixamoMapper (Python)	Traducción de Landmarks a Rig	Traduce las coordenadas de MediaPipe a un vector normalizado de 12 articulaciones (vectores unitarios 3D). Aísla la fisionomía del usuario, impidiendo que su altura o peso sesguen la evaluación.
PyTorch (LSTM)	Modelado Secuencial	La celda LSTM retiene la dinámica temporal y las transiciones secuenciales del baile de forma nativa, a diferencia de clasificadores que aplanan las secuencias.
PyQt6 (Python UI)	Interfaz Gráfica de Usuario	Plataforma moderna con diseño oscuro para controlar cámara, grabaciones, entrenamiento y evaluación sin dependencias web lentas.
Unity 3D	Motor Gráfico de Renderizado	Proporciona un entorno interactivo para reproducir las danzas sobre rigs de Mixamo y albergar la UI de retroalimentación en tiempo real.

3. Arquitectura del Sistema y Puente de Red (UDP Bridge)

Dado que el procesamiento de Machine Learning y visión artificial corre en **Python** y el renderizado interactivo ocurre en **Unity**, el sistema implementa un puente de red bidireccional asíncrono basado en el protocolo **UDP (User Datagram Protocol)** a través del **Puerto 5005**.

Flujo del pipeline: [Cámara RGB] → [YOLOv8] → [MediaPipe Pose] → [MixamoMapper] → [LiveEvaluator / MixamoAnimator.cs]

Canales UDP de Comunicación

- **Pose Stream (Envío a Unity):** Python transmite de forma continua un JSON con las orientaciones de los 12 huesos de Mixamo (ej: mixamorig:LeftArm, mixamorig:RightArm). El componente MixamoAnimator.cs parsea este paquete a 30 FPS y rota los huesos del avatar para replicar la pose en pantalla.
- **Scoring Stream (Envío a Unity):** Cuando el modo de **Evaluación en Vivo** está activo, el hilo matemático de Python calcula similitudes coseno de los 5 segmentos críticos (brazo izquierdo, brazo derecho, pierna izquierda, pierna derecha y torso). Transmite estas calificaciones mediante llaves reservadas especiales (__score_overall__, __score_torso__, etc.), capturadas por MovementComparator.cs para alimentar los indicadores en pantalla.

4. Manual de Instalación y Configuración del Entorno (Python)

Sigue estos pasos en Windows para configurar el entorno de ejecución de Python:

Paso 1: Clonar y Acceder al Directorio

Abre una terminal de PowerShell en Windows y navega al directorio del proyecto:

```
cd c:\Users\danie\Desktop\tareas\uabc\labtech\LABITO\motion_ml_app
```

Paso 2: Crear el Entorno Virtual (venv)

Aísla las dependencias del proyecto de tu sistema global de Python:

```
python -m venv venv
```

Paso 3: Activar el Entorno Virtual

```
.\venv\Scripts\activate
```

Paso 4: Instalar Dependencias

```
pip install -r requirements.txt
```

Paso 5: Validar el Entorno

Ejecuta el script de verificación automatizado para confirmar que todas las dependencias y archivos de modelos están instalados:

```
python check_env.py
```

5. Manual de Operación de la Interfaz (PyQt6)

python main.py

Se abrirá una interfaz gráfica oscura con iconos vectoriales de alta resolución. A continuación se detalla cada panel:

A. Panel de Carga y Visualización (Lado Izquierdo)

- **Cargar Video:** Permite importar un archivo de video local (.mp4, .avi, .mov) para analizarlo o preprocesarlo.
- **Usar Cámara (Toggle Inteligente):** Al hacer clic, enciende el flujo de tu cámara web por defecto. El botón cambia dinámicamente a **Detener Cámara**. Vuelve a hacer clic para apagar el sensor y liberar la cámara web.
- **Selector de Cámara (Desplegable):** Si posees múltiples cámaras conectadas, selecciona el índice del dispositivo correspondiente. Pulsa el botón de actualizar lateral para volver a escanear los puertos.

B. Grabación de Secuencias de Referencia

1. Escribe el nombre del movimiento coreográfico en el cuadro **Nombre del baile** (ej: Secuencia1).
2. Haz clic en el botón unificado **REC**. El botón se transformará a **STOP** en color naranja, se habilitará el botón **CANCELAR** al lado derecho y se encenderá un indicador parpadeante REC rojo al lado del título superior.
3. Si la grabación sale mal, presiona **CANCELAR** para vaciar el búfer de memoria y descartar los frames. Al finalizar con éxito, pulsa **STOP** para guardar la secuencia en data/sequences/.
4. **Bote de Basura (Eliminación Segura):** Al lado del selector de secuencias, un botón con el icono de un bote de basura rojo y blanco permite eliminar grabaciones. Para prevenir eliminaciones por error, al pulsarlo se abrirá un cuadro de confirmación (Sí/No) y posteriormente requerirá escribir exactamente el nombre de la secuencia como clave de doble seguridad.

C. Inteligencia Artificial, Evaluación y Logs

- **Entrenar Modelo:** Ejecuta el aprendizaje offline de las secuencias JSON grabadas. Al finalizar, guarda los pesos en data/motion_model.pt y data/label_map.pt.
- **Reproducir (LSTM):** Transmite a Unity la pose inferida por el modelo de IA recurrente.
- **Secuencia Exacta:** Envía la pose grabada frame a frame sin procesamiento de IA, ideal para calibrar y comprobar el funcionamiento físico del avatar en Unity.
- **Evaluar en Vivo:** Inicia la comparativa instantánea de tu silueta frente a la referencia cargada. Muestra tu puntuación general en pantalla y transmite las calificaciones por articulación a Unity.
- **Historial de Logs (Botón de Libreta):** Ubicado en el panel de **Estado** superior. Permite abrir un visor modal flotante que despliega el historial completo de eventos del sistema (inicialización, red UDP, carga de modelos y epochs de entrenamiento)

coloreado en tiempo real sin saturación de conteo de frames. El sistema cuenta con redirección nativa de consola (`sys.stdout`) conectada a señales Qt para asegurar que el registro desde subprocesos paralelos sea 100% estable (*thread-safe*).

6. Guía de Integración e Instalación en Unity 3D

Para recibir la señal y visualizar el avatar junto con los paneles de feedback dinámicos, configura los siguientes scripts C# en tu escena:

Paso 1: Configurar el Receptor UDP y Animador

1. En tu jerarquía de escena, selecciona el GameObject que contiene al avatar 3D con rig de Mixamo.
2. Asegúrate de tener el script `UDPClient.cs` agregado al GameObject. Configura el puerto de escucha en **5005**.
3. Agrega el script `MixamoAnimator.cs` al mismo GameObject. Leerá los datos de articulación recibidos y los aplicará al esqueleto en tiempo real.

Paso 2: Configurar la Comparación de Movimientos

1. En el mismo GameObject del personaje, agrega el componente `MovementComparator.cs`.
2. Este script intercepta y parsea las llaves especiales de puntuación de red (`__score__`).
3. Define los límites de color en el Inspector (ej: verde para precisiones >75%, amarillo entre 50% y 74%, rojo para precisión <50%).

Paso 3: Configurar el Canvas de Retroalimentación (Feedback UI)

1. Crea un Canvas en tu escena (GameObject → UI → Canvas).
2. Agrega el script `FeedbackUI.cs` a un objeto vacío dentro del Canvas.
3. En el Inspector, asigna las referencias visuales:
 - **Comparator:** Arrastra el GameObject del personaje con `MovementComparator.cs`.
 - **Score Global (TextMeshPro):** Elemento de texto que mostrará el porcentaje global en tiempo real (ej: 87%).
 - **Paneles de Segmento (Image):** Arrastra las imágenes del Canvas correspondientes a brazo izquierdo, brazo derecho, pierna izquierda, pierna derecha y torso. El script interpolará los colores entre verde, amarillo y rojo según la precisión.
4. Entra en **Play Mode** en Unity para comenzar a recibir la señal interactiva de Python.

7. Resolución de Problemas Comunes (Troubleshooting)

A. OpenCV no puede abrir la cámara (Warning cap.cpp)

- **Síntoma:** Al arrancar el programa, aparecen múltiples advertencias en consola referentes a VideoCapture VIDEOIO(DSHOW).
- **Causa:** OpenCV está escaneando los controladores del sistema para identificar cámaras USB. Es un comportamiento informativo de Windows, completamente inofensivo.
- **Solución:** Si tu cámara no emite imagen, asegúrate de que ninguna otra aplicación (Zoom, Teams, Discord) la esté utilizando. Cambia el índice de cámara en el desplegable de PyQt6 y presiona el botón de actualizar.

B. El avatar 3D en Unity no se mueve (UDP desconectado)

- **Síntoma:** Python muestra el flujo de la cámara correctamente, pero el personaje en Unity se queda inmóvil.
- **Causa:** Hay un bloqueo de socket de red, o los puertos no coinciden.
- **Solución:**
 1. Verifica que tu cortafuegos (Windows Firewall) permita conexiones entrantes locales en el puerto 5005.
 2. Asegúrate de tener Unity en **Play Mode**. La retransmisión no se reflejará si la escena está en pausa.
 3. Comprueba en la consola de Unity si hay errores de parseo de red o si el puerto configurado en UDPClient.cs es exactamente 5005.

C. La evaluación en vivo marca puntuaciones muy bajas (Descalibración)

- **Síntoma:** Realizas el movimiento correctamente, pero la silueta en Unity se colorea constantemente en rojo.
- **Causa:** Estás demasiado cerca o lejos del encuadre, o la iluminación impide que MediaPipe distinga tus extremidades.
- **Solución:**
 4. Mantén una distancia de entre 2 y 3 metros del lente, garantizando que tus pies y brazos sean visibles en el cuadro.
 5. Evita vestir ropa excesivamente holgada o de colores idénticos al fondo de tu habitación.
 6. Asegúrate de que no haya oclusiones físicas (sillas, escritorios, cables) obstruyendo la trayectoria visual de tus piernas y torso hacia la webcam.